INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE RONDÔNIA

Tecnologia em Sistemas para Internet

InventariAR

Roteiro de estudo e explicação do projeto

Atividade 2 - Desenvolvimento Web III + Banco de Dados II

Identificação	Informação
Ano da atividade	2026
Equipe	Rodolfo Viana e Yngride Gabrielle Nascimento Santos
Responsabilidade e revisão	Distribuição confirmada pelos integrantes; quadro na seção 7
Versão do material	Adaptação local para estudo e apresentação

Importante: este material explica o código que existe neste workspace. A equipe confirmou a distribuição de tarefas apresentada na seção 7. Este é um material de estudo e explicação, com as capturas dos testes realizadas.

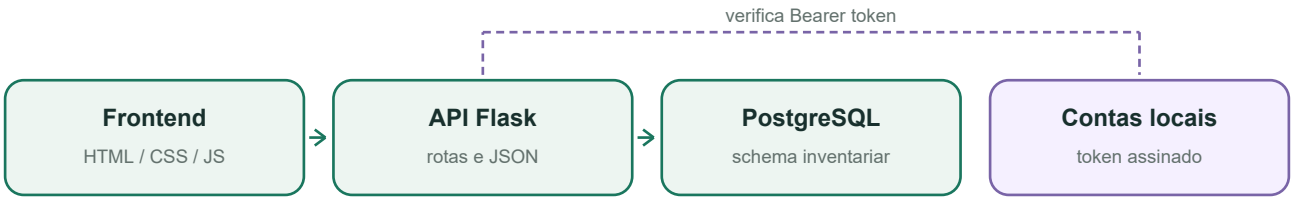
1. O que o sistema faz

O InventariAR organiza equipamentos de patrimônio. Cada registro guarda tombamento, nome, categoria, local atual, condição e status. A interface local mostra um painel, lista pesquisável, cadastro, edição, exclusão e movimentação. As operações de inventário passam pela API Flask em JSON e ficam no PostgreSQL.

A primeira versão visual do projeto foi criada no Lovable. Depois, a interface foi trazida para este workspace e reconstruída localmente em HTML, CSS e JavaScript para funcionar com a API Flask e o PostgreSQL configurados aqui. Esta versão não é uma exportação completa do projeto Lovable e não inclui os recursos de câmera ou realidade aumentada da versão original.

Como explicar em voz alta: “O objetivo é registrar os equipamentos e manter a localização e a condição atualizadas. O navegador não acessa o banco diretamente: ele envia requisições para a API, e a API aplica as validações e grava os dados no PostgreSQL.”

2. Arquitetura



Setas indicam as trocas principais; a API valida o usuário antes das rotas protegidas.

O frontend envia requisições HTTP para a API Flask. O usuário cria uma conta local no PostgreSQL; a senha é armazenada somente como hash. A API entrega um token assinado e verifica esse token nas rotas protegidas, associando o identificador do usuário à movimentação. Nenhum serviço externo de autenticação é necessário.

GET /api/health verifica o acesso ao PostgreSQL. O cadastro e o login locais emitem um token assinado, com validade de 12 horas, exigido pelas rotas de inventário. A chave de assinatura fica no arquivo local de ambiente e não é versionada.

3. Banco de dados e integridade

O script principal é backend/sql/001_schema.sql. O modelo separa os dados em cinco tabelas para evitar repetir nomes de categoria e local em cada equipamento, manter as contas locais e registrar o histórico de alterações.

Tabela	Função e relações
categorias	Lista os nomes de categoria. A PK id é referenciada por equipamentos.
locais	Lista os nomes de local. A PK id é referenciada por equipamentos e movimentações.
equipamentos	Cadastro principal. Tem PK id, tombamento único e FKs categoria_id e local_id.
usuarios	Contas locais com nome, e-mail único, hash de senha e identificador UUID.
movimentacoes	Histórico ligado ao equipamento, com origem/destino, valores anteriores e novos, motivo, autor e data.

PK identifica cada linha. FK aponta para uma linha relacionada e impede referências inexistentes. UNIQUE em tombamento evita dois equipamentos com o mesmo código patrimonial. Os índices apoiam filtros, relações e consultas do histórico.

As regras CHECK incluem nomes e tombamento não vazios, status limitado a ativo/inativo, condição limitada a cinco valores, motivo obrigatório e validação dos valores anterior/novo no histórico. Existem mais de três CHECKs

no script.

3.1 View e stored procedures

A view inventariar.vw_patrimonio combina equipamentos, categorias e locais por JOIN. Assim, a API e o relatório podem ler nomes compreensíveis sem duplicar esses nomes na tabela de equipamentos.

A procedure inventariar.registrar_movimentacao recebe equipamento, local, status, condição, motivo e autor. Ela bloqueia a linha do equipamento durante a transação, lê os valores anteriores, atualiza os valores atuais e grava o histórico. A rota POST de movimentações chama essa procedure.

A procedure inventariar.gerar_relatorio abre um cursor consultando a view, agrupando quantidade por categoria, local, condição e status. Ela é executada no próprio banco; o enunciado não exige que a API a chame. O exemplo comentado no SQL mostra BEGIN, CALL, FETCH e COMMIT porque o cursor só existe durante a transação.

4. API e contrato JSON

Método e rota	Comportamento
GET /api/health	Confirma que a API consegue consultar o PostgreSQL.
POST /api/auth/register	Cria uma conta local e devolve um token assinado.
POST /api/auth/login	Confere e-mail/senha e inicia a sessão local.
GET /api/equipamentos	Lista; aceita busca e filtros validados.
GET /api/equipamentos/<id>	Consulta um registro.
POST /api/equipamentos	Cria e retorna HTTP 201.
PUT ou PATCH /api/equipamentos/<id>	Atualiza o cadastro; mudanças de local/status/condição também geram histórico.
DELETE /api/equipamentos/<id>	Exclui o equipamento e, pela FK com cascade, seu histórico.
POST /api/equipamentos/<id>/movimentacoes	Chama a procedure e retorna o estado atualizado.
GET /api/equipamentos/<id>/movimentacoes	Lista o histórico do equipamento.
GET /api/relatorios/patrimonio	Agrupa os registros lidos pela view.

A API confere o formato e os campos de entrada antes de abrir a operação no banco. Textos são limitados, valores fora das listas permitidas são rejeitados e os erros voltam como JSON com código HTTP. A transação faz commit se a operação termina e rollback quando ocorre uma exceção.

Exemplo do corpo para criar um registro:

```
{
  "nome": "Notebook de teste",
  "tombamento": "IFRO-TESTE-001",
  "categoria": "Informática",
  "local": "Laboratório 2",
  "status": "ativo",
  "condicao": "bom"
}
```

4.1 Como explicar uma requisição da interface até o banco

Um exemplo simples é cadastrar um equipamento. A tela transforma os campos preenchidos em JSON e chama a função api() de frontend/app.js. Essa função usa fetch() para enviar uma requisição HTTP à API. No cadastro, o método é POST e o caminho é /api/equipamentos.

Etapa	O que acontece no projeto
1. Interface	frontend/app.js transforma os valores do formulário em JSON e chama api() com o método POST.
2. Rota	O Flask encontra @api.post('/equipamentos') em backend/inventariar/routes.py.
3. Autenticação e validação	O decorador @protected confere o token; body() lê JSON e validate_equipment() verifica os campos.
4. Persistência	connection() abre uma transação PostgreSQL; comandos SQL parametrizados salvam os dados nas tabelas.
5. Resposta	O Flask devolve JSON e HTTP 201; a interface recarrega a lista para mostrar o registro persistido.

O símbolo @ é um decorador do Flask. Por exemplo, @api.post('/equipamentos') registra qual função atende uma requisição POST nesse caminho. O decorador @protected executa a autenticação antes da função da rota. Isso evita repetir essa verificação dentro de cada endpoint protegido.

Os valores SQL são passados separadamente do texto da consulta, usando marcadores %s e uma tupla de parâmetros. Assim, os dados recebidos não são concatenados diretamente no comando SQL. Já os nomes de tabela dinâmicos usados para categorias e locais passam por sql.Identifier, do psycopg.

A função connection() usa o contexto da conexão para finalizar a transação: se o bloco termina normalmente, há commit; se ocorre uma exceção, há rollback. Erros conhecidos do PostgreSQL são convertidos em respostas da API, como 409 para valor único duplicado, 422 para dados inválidos e 503 quando o banco está indisponível.

Como explicar uma movimentação

Na movimentação, a interface envia POST para /api/equipamentos/<id>/movimentacoes. A rota valida local, condição, status e motivo. Em seguida, a aplicação chama CALL inventariar.registrar_movimentacao(...). A procedure no banco atualiza o equipamento e grava o histórico na mesma transação. Por isso, a resposta 201 só é enviada depois que a operação termina com sucesso.

A rota também bloqueia a linha do equipamento durante a alteração com SELECT ... FOR UPDATE. Esse bloqueio ajuda a evitar que duas alterações concorrentes leiam e sobrescrevam o mesmo estado ao mesmo tempo.

Autenticação em palavras simples

No cadastro, a API valida os dados e grava o hash da senha, não a senha original. No login, compara a senha informada com o hash guardado. Se estiver correta, emite um token assinado. O frontend envia esse token no cabeçalho Authorization: Bearer ... nas rotas protegidas. A API valida assinatura e prazo; a validade é de 12 horas.

Um roteiro curto para apresentar

“A interface envia JSON por HTTP para as rotas do Flask. Cada rota confere a autenticação e valida os campos antes de acessar o banco. As consultas são parametrizadas e executadas em uma transação PostgreSQL. Nas movimentações, a API chama uma procedure que atualiza o equipamento e registra o histórico. No fim, a API devolve JSON com um código HTTP, e a tela atualiza os dados.”

Perguntas comuns sobre a API

Pergunta	Resposta que você pode explicar
Qual a diferença entre GET e POST?	GET consulta dados; POST envia dados para criar algo ou executar uma ação, como cadastrar ou movimentar.
O que significa HTTP 201?	A requisição foi atendida e um novo registro foi criado.
Por que a senha não é salva diretamente?	O banco guarda um hash; no login, a senha digitada é comparada usando a função própria de verificação.
O que acontece se a validação falhar?	A API não grava os dados e retorna uma mensagem JSON com código HTTP apropriado, como 422.
Por que usar uma procedure na movimentação?	Para manter a atualização do equipamento e o registro do histórico juntos no banco.

5. Frontend e configuração

A página inicial está em frontend/index.html; o comportamento fica em frontend/app.js e o visual em frontend/styles.css. O arquivo frontend/config.js aponta para a URL local da API e é ignorado pelo Git. Não há URLs externas de autenticação para configurar.

O backend lê DATABASE_URL, SECRET_KEY e CORS_ORIGINS de backend/.env. O configurador gera a chave privada de sessão. As contas ficam na tabela inventariar.usuarios; a senha original nunca é gravada. O frontend não precisa de chave secreta ou conta de autenticação externa.

O token de acesso assinado é guardado em sessionStorage durante a sessão atual do navegador e enviado no cabeçalho Authorization. A sessão expira após 12 horas. A interface não usa registros fictícios como resposta alternativa: se a API falhar, mostra a mensagem de erro.

Como executar com o banco local configurado:

```
# Terminal 1 - dentro de backend, com o ambiente virtual ativado
python run.py

# Terminal 2 - na raiz do workspace
python -m http.server 8080 --directory frontend

# Abrir no navegador
http://localhost:8080
```

6. Plano de testes para demonstração

Os testes automatizados verificam validações, tokens e respostas JSON. Também foi realizado um teste integrado no PostgreSQL local: cadastro/login, senha incorreta, cadastro de equipamento, movimentação pela procedure e relatório. Os registros temporários foram removidos ao final. Resultado da suíte automatizada: 10 testes passaram.

Escopo das evidências: as capturas a seguir foram feitas durante o teste do frontend local e durante requisições HTTP reais para a API Flask conectada ao PostgreSQL. Os dados usados para a demonstração são temporários; senhas e tokens não aparecem nas imagens.

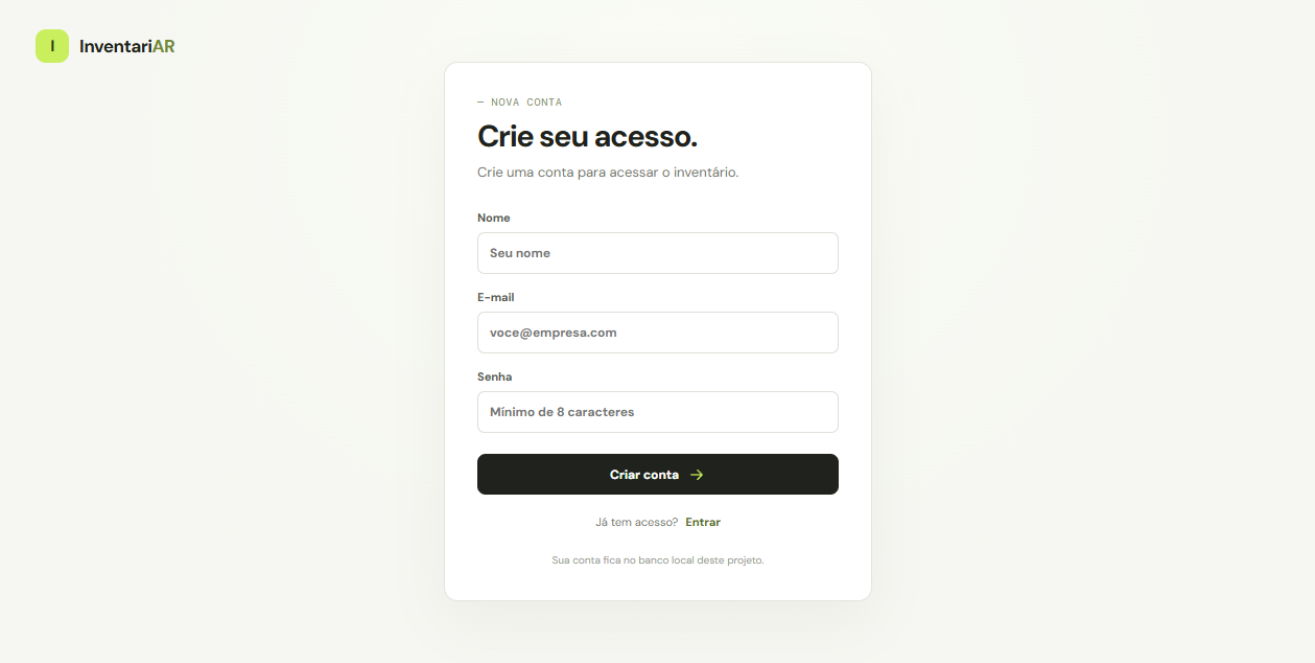
Teste real	Endpoint	Resultado
Saúde do serviço e conexão com PostgreSQL	GET /api/health	HTTP 200; status ok.
Criar equipamento	POST /api/equipamentos	HTTP 201.
Consultar e atualizar equipamento	GET e PUT /api/equipamentos/<id>	HTTP 200.
Registrar movimentação pela procedure	POST /api/equipamentos/<id>/movimentacoes	HTTP 201.
Consultar histórico e relatório	GET /api/equipamentos/<id>/movimentacoes; GET /api/relatorios/patrimonio	HTTP 200.
Tentar repetir tombamento	POST /api/equipamentos	HTTP 409 esperado; duplicidade recusada.
Excluir registro temporário	DELETE /api/equipamentos/<id>	HTTP 200; registro removido.

As telas de cadastro, movimentação, edição e exclusão foram exercitadas pela interface. Os métodos, endpoints e códigos HTTP abaixo foram registrados em requisições reais feitas no navegador. A resposta 409 para tombamento repetido é um bloqueio esperado, não uma falha do teste.

6.1 Capturas dos testes

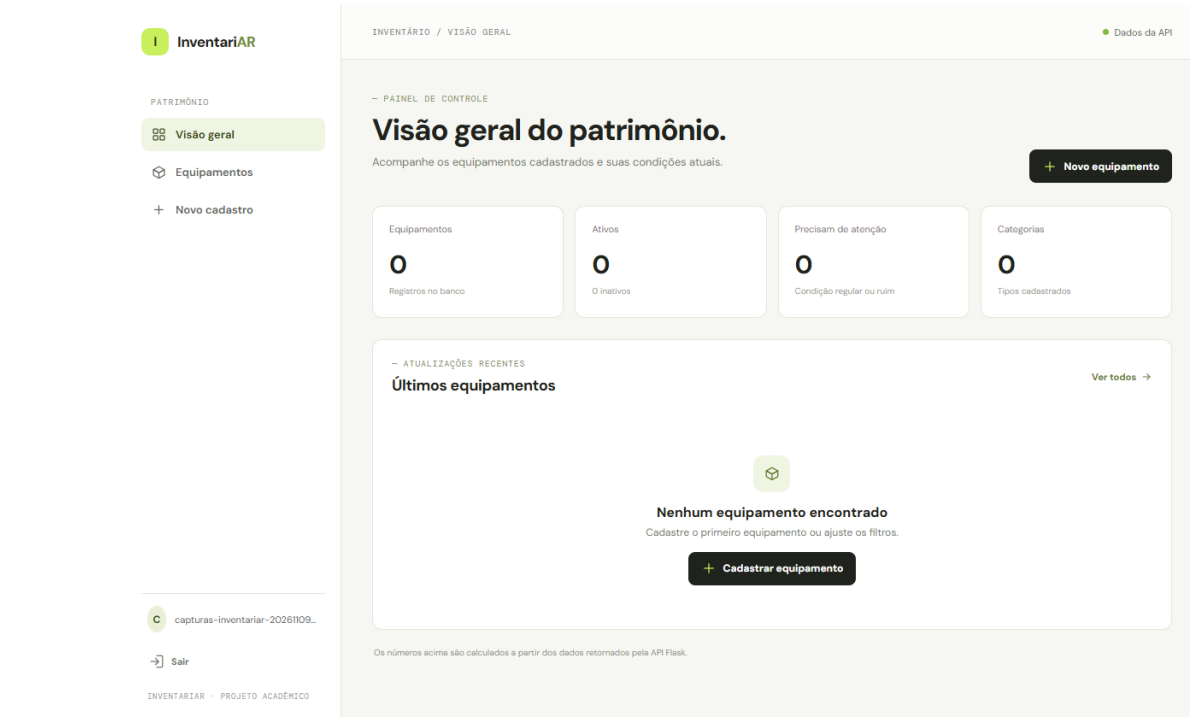
Cada figura tem uma legenda para identificar a tela ou requisição testada. As capturas foram feitas durante os testes no ambiente local.

Figura 1. Criação de conta local



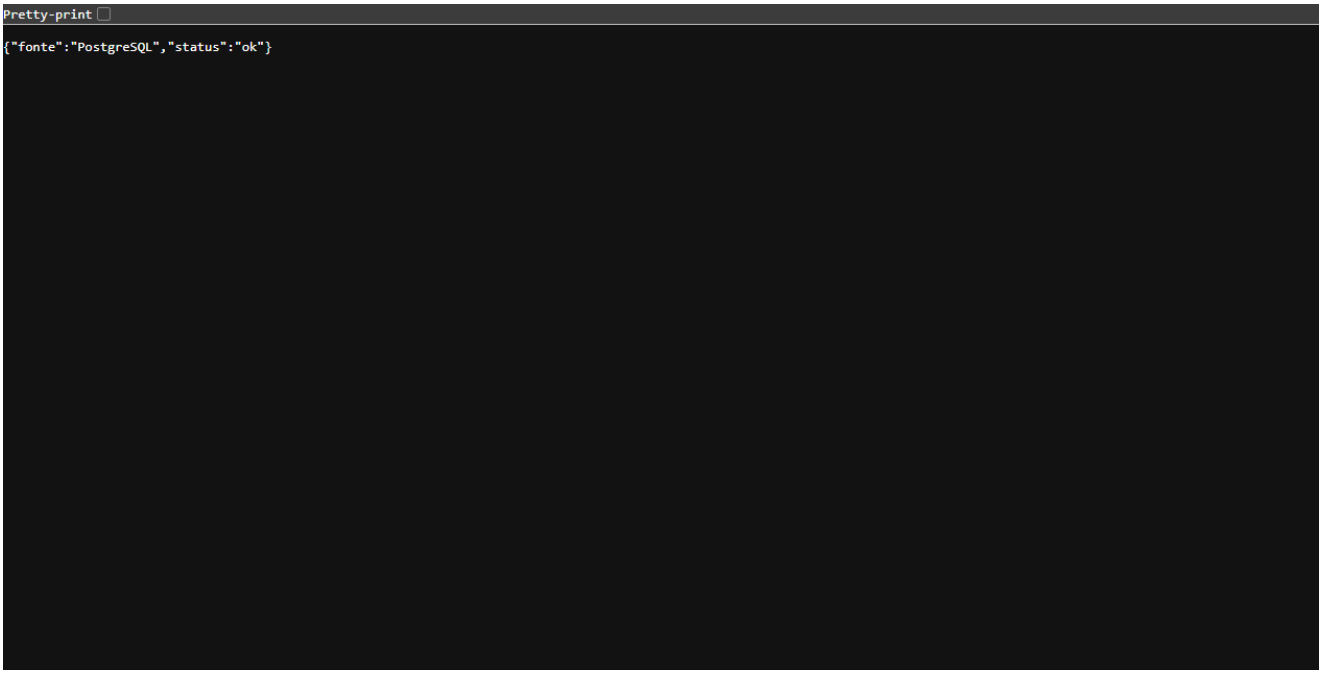
Tela usada para cadastrar uma conta de teste; a conta temporária foi removida ao concluir a coleta de evidências.

Figura 2. Painel após autenticação



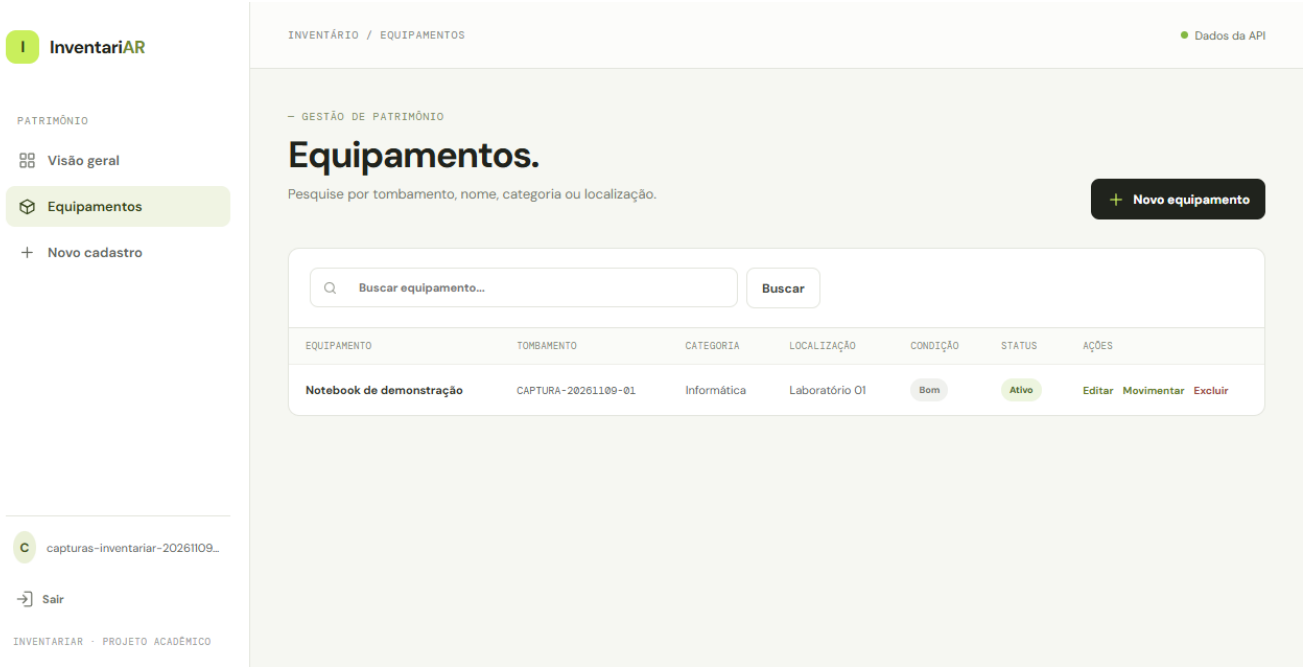
Painel carregado depois do cadastro local; os indicadores usam a resposta da API.

Figura 3. Verificação da API e do PostgreSQL



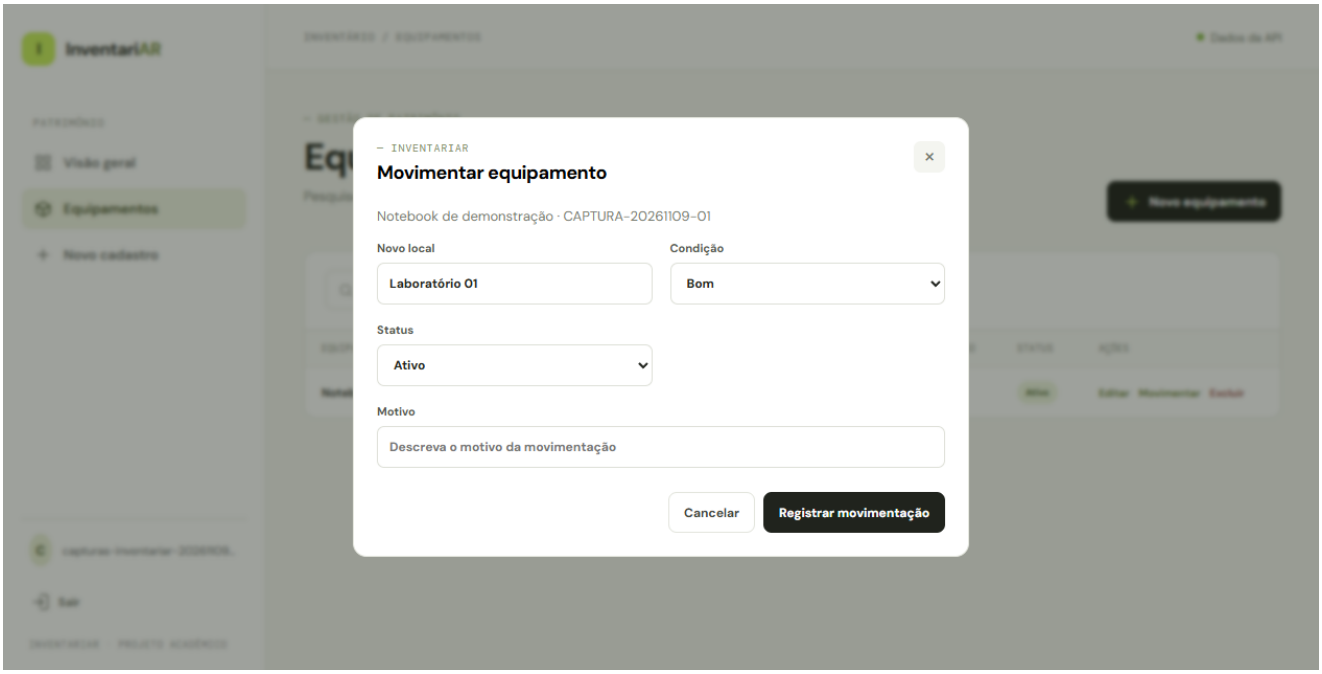
GET /api/health respondeu HTTP 200 com status ok e fonte PostgreSQL.

Figura 4. Leitura do equipamento cadastrado



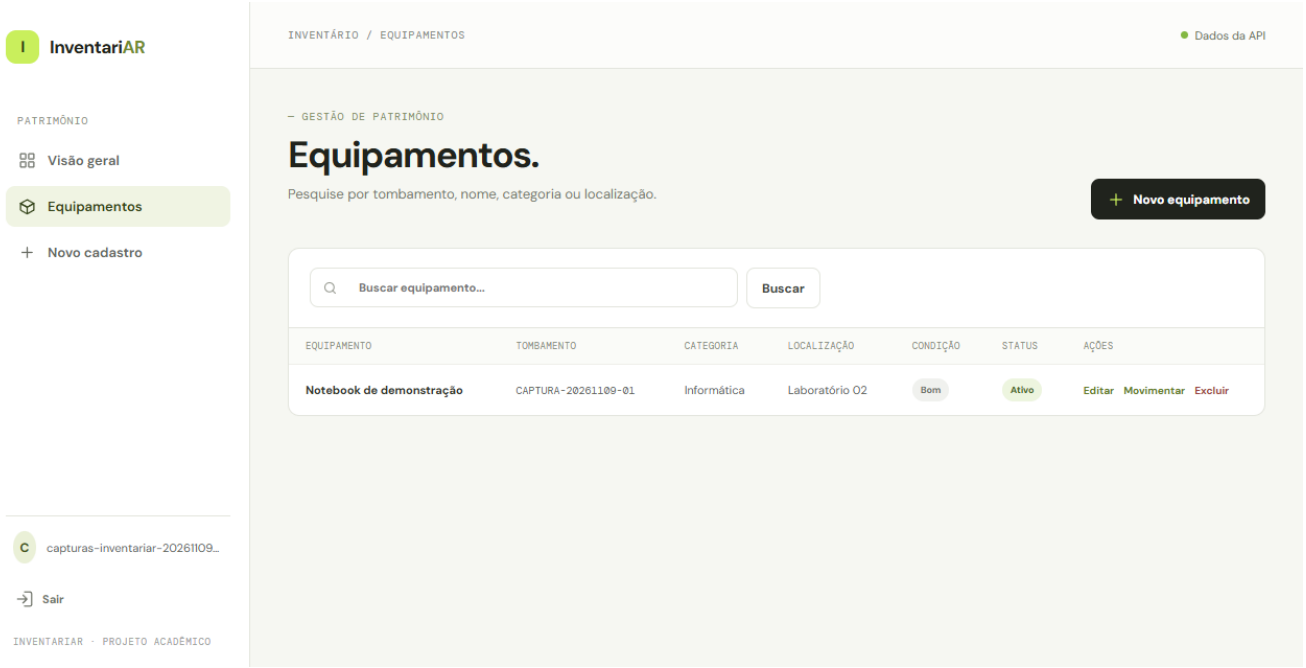
Equipamento criado pela interface e apresentado na listagem; operação POST retornou HTTP 201.

Figura 5. Formulário de movimentação



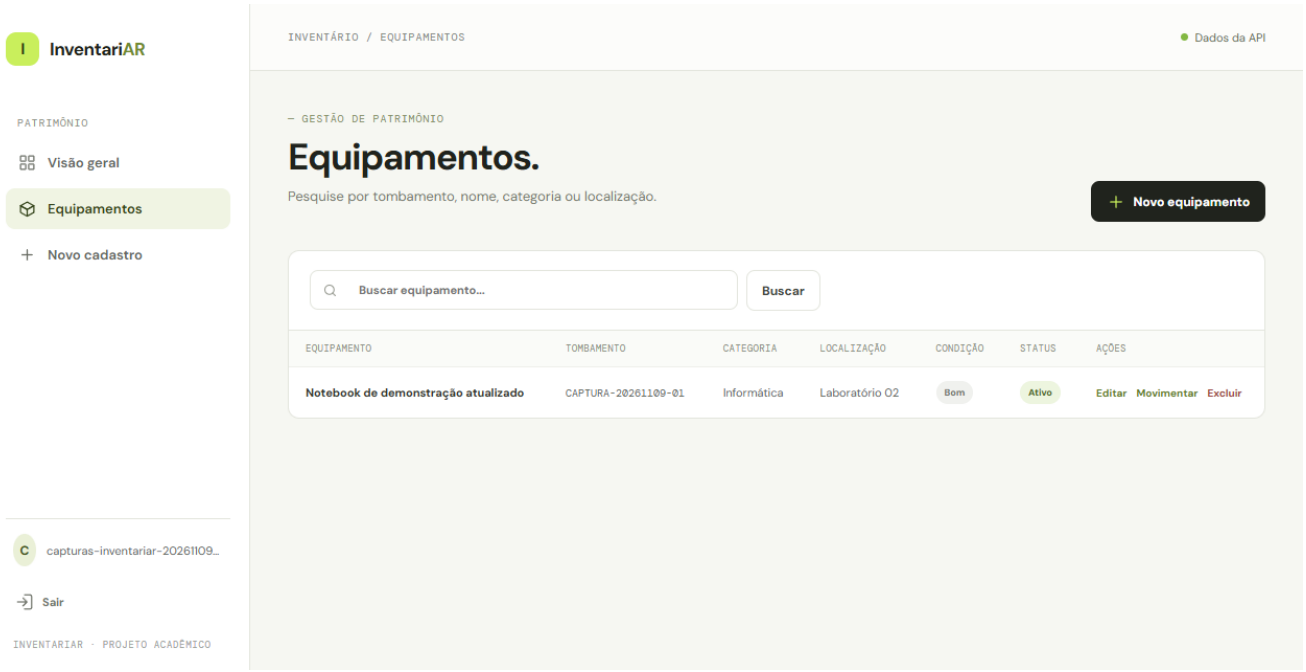
Formulário preenchido para transferir o equipamento de teste; a ação chama a procedure no banco.

Figura 6. Tela de equipamentos após os testes



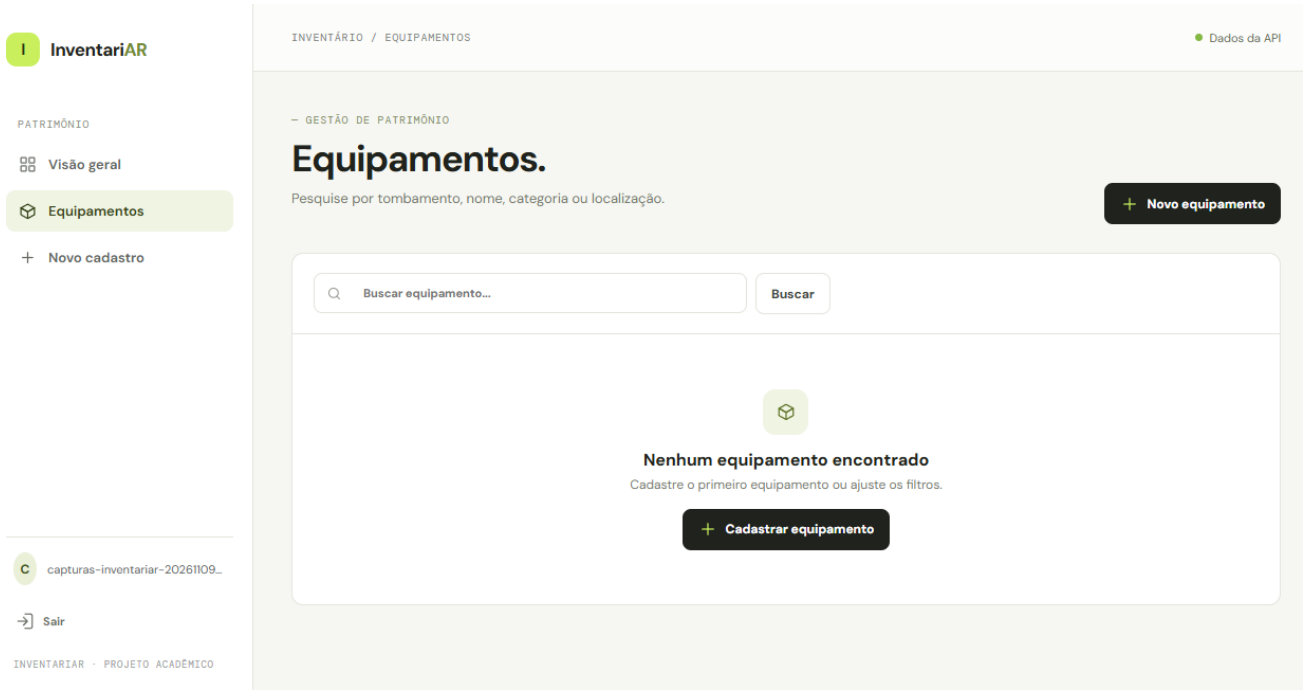
A listagem está vazia após a limpeza do registro temporário. A movimentação pela procedure retornou HTTP 201, conforme o resumo da figura 9.

Figura 7. Edição do cadastro



Nome atualizado pela interface e valor persistido na listagem; operação PUT retornou HTTP 200.

Figura 8. Exclusão do registro de teste



A listagem fica vazia após excluir o equipamento temporário; operação DELETE retornou HTTP 200.

Figura 9. Resumo das requisições HTTP reais



InventariAR — evidências dos testes da API

Requisições HTTP reais à API Flask local, conectada ao PostgreSQL. Os dados usados foram temporários e removidos ao final.

Ambiente local · execução pelo navegador

Método	Endpoint	Teste	HTTP	Resultado
POST	/api/equipamentos	Criar equipamento	201	OK
GET	/api/equipamentos/4	Consultar equipamento	200	OK
PUT	/api/equipamentos/4	Atualizar equipamento	200	OK
POST	/api/equipamentos/4/movimentacoes	Registrar movimentação (procedure)	201	OK
GET	/api/equipamentos/4/movimentacoes	Consultar histórico	200	OK
GET	/api/relatorios/patrimonio	Relatório de patrimônio	200	OK
POST	/api/equipamentos	Rejeitar tombamento duplicado	409	Bloqueio esperado
DELETE	/api/equipamentos/4	Excluir equipamento temporário	200	OK

A senha e o token de acesso não são exibidos. HTTP 409 no teste de duplicidade é o resultado esperado: o registro não deve ser repetido.

Resumo exibido no navegador a partir das respostas reais da API: CRUD, movimentação, histórico, relatório e conflito esperado para tombamento duplicado.

7. Divisão e dupla revisão

A atividade pede que cada parte tenha um responsável e seja conferida por pelo menos outra pessoa. A equipe confirmou a seguinte distribuição para este roteiro:

Parte	Responsável	Conferente (dupla revisão)
Modelo relacional, CHECKs, view e procedures	Rodolfo Viana	Yngride Gabrielle Nascimento Santos
API Flask, rotas JSON e validações	Yngride Gabrielle Nascimento Santos	Rodolfo Viana
Interface e integração	Rodolfo Viana	Yngride Gabrielle Nascimento Santos
Documentação, testes e compactação	Yngride Gabrielle Nascimento Santos	Rodolfo Viana

8. Perguntas que podem aparecer

Pergunta	Resposta curta
Por que separar categoria e local?	Para reduzir repetição e manter referências consistentes com chaves estrangeiras.
O que a procedure de movimentação garante?	Atualiza o equipamento e registra os valores antigos e novos na mesma transação, com bloqueio contra alterações concorrentes.
Por que a API devolve JSON?	JSON é o formato de troca usado entre a interface e os endpoints REST.
Onde está o CRUD?	Nas rotas de equipamentos: POST cria, GET consulta, PUT/PATCH atualiza e DELETE exclui.
A procedure de relatório é chamada pela API?	Não. Ela existe no banco e consulta a view, como permite o enunciado.

Pergunta	Resposta curta
A integração foi testada de ponta a ponta?	Sim, localmente: cadastro/login, CRUD, procedure de movimentação e relatório foram testados com PostgreSQL.

Checklist antes de entregar: revisar se a divisão da seção 7 corresponde ao trabalho realizado; confirmar com a professora o uso do banco local; gerar o documento final único e incluir script SQL e código-fonte no ZIP com o nome solicitado.